Kathmandu University

Department of Computer Science and Engineering

Dhulikhel, Kavre

Lab Report 3

[Course Code: COMP 307]

Submitted by:
Jatin Madhikarmi
Roll No. 32

Submitted to:
Ms. Rabina Shrestha
Department of Computer Science and Engineering

1 Introduction

CPU scheduling is the process by which the operating system decides which process in the ready queue is to be allocated the CPU. These scheduling schemes are generally categorized into two types: **Preemptive** and **Non-preemptive**.

—

Preemptive Scheduling

In preemptive scheduling, the OS can interrupt a currently running process and move it back to the ready queue to allocate the CPU to another process (usually one with higher priority).

- **Mechanism:** The execution of a process can be stopped mid-way.
- **Flexibility:** Highly flexible as it allows for time-sharing and real-time processing.
- **Overhead:** Higher overhead due to frequent *context switching*.
- **Examples:** Round Robin (RR), Shortest Remaining Time First (SRTF), Priority (Preemptive).

Non-preemptive Scheduling

In non-preemptive scheduling, once a process starts executing, it holds the CPU until it either terminates or switches to a waiting state (e.g., for I/O).

- **Mechanism:** No external interruption can force a process to give up the CPU.
- **Predictability:** It is simple and predictable but can lead to the *Convoy Effect*.
- **Overhead:** Lower overhead because context switches occur only when a process finishes.
- **Examples:** First Come First Served (FCFS), Shortest Job First (SJF).

—

Comparison Summary

Table 1: Key Differences

Feature	Preemptive	Non-preemptive
Interruption	Can be interrupted	Cannot be interrupted
Data Integrity	Risks shared data concurrency	More secure for shared data
Overhead	High (Context Switching)	Low
Response Time	Fast	Slow
Starvation	Possible for low priority	Possible for long processes

2 Introduction to SJF Scheduling

Shortest Job First (SJF) is a non-preemptive scheduling algorithm that prioritizes processes based on their execution time. It is considered optimal because it minimizes the average waiting time for a given set of processes by ensuring that shorter tasks are completed before longer ones, reducing the overall ”queue” build-up.

Working Mechanism

The algorithm operates based on the following logic:

1. When the CPU is free, the scheduler checks the **Ready Queue** for all available processes.
2. It selects the process with the smallest **Burst Time** (the time required for execution).
3. If two processes have identical burst times, the **First Come First Served (FCFS)** rule is typically applied as a tie-breaker.
4. In the non-preemptive version, once a process begins execution, it holds the CPU until it completes its entire burst cycle.

Turnaround Time Calculation

Turnaround Time (TAT) is the total interval from the time of submission of a process to the time of its completion. To calculate it, we first need the **Completion Time** (CT), which is the time at which the process finishes.

The formula for Turnaround Time is:

$$TAT = CT - AT \tag{1}$$

Where:

- **CT**: Completion Time

- **AT**: Arrival Time

Additionally, the **Waiting Time** (WT) can be derived from the Turnaround Time as follows:

$$WT = TAT - BT \tag{2}$$

Where **BT** is the Burst Time of the process.

Program Code

```c
#include <stdio.h>

struct Process
{
    int pid;
    int bt;  // Burst Time
    int wt;  // Waiting Time
    int tat; // Turnaround Time
};

void sortProcesses(struct Process p[], int n)
{
    struct Process temp;
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (p[j].bt > p[j + 1].bt)
            {
                temp = p[j];
                p[j] = p[j + 1];
                p[j + 1] = temp;
            }
        }
    }
}
```

Figure 1: Implemenation of SJF Algorithm Source Code 1

```c
int main()
{
    int n;
    float totalWT = 0, totalTAT = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    struct Process p[n];

    for (int i = 0; i < n; i++)
    {
        p[i].pid = i + 1;
        printf("Enter burst time for Process %d: ", p[i].pid);
        scanf("%d", &p[i].bt);
    }

    // Sort by Burst Time (The core of SJF)
    sortProcesses(p, n);

    // Calculate Waiting Time and Turnaround Time
    p[0].wt = 0; // First process doesn't wait
    for (int i = 1; i < n; i++)
    {
        p[i].wt = p[i - 1].wt + p[i - 1].bt;
    }

    for (int i = 0; i < n; i++)
    {
        p[i].tat = p[i].bt + p[i].wt;
        totalWT += p[i].wt;
        totalTAT += p[i].tat;
    }
```

Figure 2: Implemenation of SJF Algorithm Source Code 2

```c
    // Display Table
    printf("\nPID\tBT\tWT\tTAT\n");
    for (int i = 0; i < n; i++)
    {
        printf("%d\t%d\t%d\t%d\n", p[i].pid, p[i].bt, p[i].wt, p[i].tat);
    }

    // Gantt Chart
    printf("\n--- Gantt Chart ---\n ");
    for (int i = 0; i < n; i++)
        printf("------- ");
    printf("\n|");
    for (int i = 0; i < n; i++)
        printf("  P%d  |", p[i].pid);
    printf("\n ");
    for (int i = 0; i < n; i++)
        printf("------- ");

    printf("\n0");
    int current_time = 0;
    for (int i = 0; i < n; i++)
    {
        current_time += p[i].bt;
        printf("      %d", current_time);
    }

    printf("\n\nAverage Waiting Time: %.2f", totalWT / n);
    printf("\nAverage Turnaround Time: %.2f\n", totalTAT / n);

    return 0;
}
```

Figure 3: Implementation of SJF Algorithm Source Code 3

```
jatin@JatinMadhikarmi:/mnt/c/Users/jatin/OneDrive/Desktop/COMP 307 (OS)/Programs$ cd "/mnt/c/Users/jatin/OneDrive/Desktop/COMP 307 (OS)/Programs/" && gcc SJF.c -o
SJF && "/mnt/c/Users/jatin/OneDrive/Desktop/COMP 307 (OS)/Programs/"SJF
Enter number of processes: 4
Enter burst time for Process 1: 2
Enter burst time for Process 2: 3
Enter burst time for Process 3: 1
Enter burst time for Process 4: 5

PID     BT      WT      TAT
3       1       0       1
1       2       1       3
2       3       3       6
4       5       6       11

--- Gantt Chart ---
 ------- ------- ------- -------
|  P3  |  P1  |  P2  |  P4  |
 ------- ------- ------- -------
0       1       3       6       11

Average Waiting Time: 2.50
Average Turnaround Time: 5.25
```

Figure 4: Output of SJF Algorithm

3 Shortest Remaining Time First (SRTF) Scheduling

Shortest Remaining Time First (SRTF) is the **preemptive** version of the SJF algorithm. In this policy, the process with the smallest amount of time remaining until completion is selected to execute.

Working Mechanism

1. The CPU is allocated to the process that arrives first.
2. If a new process arrives with a **Burst Time** shorter than the **Remaining Time** of the current process, the CPU is preempted and allocated to the new process.
3. This process continues until all tasks are completed.
4. **Context Switching:** Unlike SJF, SRTF involves more frequent context switches as processes are interrupted mid-execution.

Performance Metrics

The calculations for Turnaround Time (TAT) and Waiting Time (WT) are identical to SJF:

$$TAT = CT - AT \tag{3}$$

$$WT = TAT - BT \tag{4}$$

However, because the process may be interrupted multiple times, the **Completion Time (**CT**)** is only recorded when the **Remaining Time (**RT**)** of a process reaches zero.

Program Code

```c
#include <stdio.h>
#include <limits.h>

struct Process
{
    int pid;
    int at;  // Arrival Time
    int bt;  // Burst Time
    int rt;  // Remaining Time
    int ct;  // Completion Time
    int tat; // Turnaround Time
    int wt;  // Waiting Time
};

int main()
{
    int n, completed = 0, current_time = 0, min_val = INT_MAX;
    int shortest = 0, finish_time;
    int found = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    struct Process p[n];
    for (int i = 0; i < n; i++)
    {
        p[i].pid = i + 1;
        printf("Enter arrival and burst time for P%d: ", p[i].pid);
        scanf("%d %d", &p[i].at, &p[i].bt);
        p[i].rt = p[i].bt; // Initialize remaining time
    }

    printf("\nGantt Chart:\n|");
```

Figure 5: Implemenation of SRTF Algorithm Source Code 1

```c
while (completed != n)
{
    min_val = INT_MAX;
    found = 0;

    for (int i = 0; i < n; i++)
    {
        if (p[i].at <= current_time && p[i].rt < min_val && p[i].rt > 0)
        {
            min_val = p[i].rt;
            shortest = i;
            found = 1;
        }
    }

    if (found == 0)
    {
        current_time++;
        continue;
    }

    // Simulating 1 unit of execution
    p[shortest].rt--;
    printf(" P%d |", p[shortest].pid);

    if (p[shortest].rt == 0)
    {
        completed++;
        finish_time = current_time + 1;
        p[shortest].ct = finish_time;
        p[shortest].tat = p[shortest].ct - p[shortest].at;
        p[shortest].wt = p[shortest].tat - p[shortest].bt;
    }
    current_time++;
}
```

Figure 6: Implemenation of SRTF Algorithm Source Code 2

```c
    printf("\n\nPID\tAT\tBT\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++)
    {
        printf("%d\t%d\t%d\t%d\t%d\t%d\n", p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);
    }

    return 0;
}
```

Figure 7: Implementation of SRTF Algorithm Source Code 3

```
jatin@JatinMadhikarmi:/mnt/c/Users/jatin/OneDrive/Desktop/COMP 307 (OS)/Programs$ cd "/mnt/c/Users/jatin/OneDrive/Desktop/COMP 307 (OS)/Programs/" && gcc SRTF.c -o SRTF && "/mnt/c/Users/jatin/OneDrive/Desktop/COMP 307 (OS)/Programs/"SRTF
Enter number of processes: 4
Enter arrival and burst time for P1: 0 3
Enter arrival and burst time for P2: 1 2
Enter arrival and burst time for P3: 2 1
Enter arrival and burst time for P4: 4 5

Gantt Chart:
| P1 | P1 | P1 | P3 | P2 | P2 | P4 | P4 | P4 | P4 | P4 |

PID     AT      BT      CT      TAT     WT
1       0       3       3       3       0
2       1       2       6       5       3
3       2       1       4       2       1
4       4       5       11      7       2
```

Figure 8: Output of SRTF Algorithm

4 Round Robin (RR) Scheduling

Round Robin is a **preemptive** scheduling algorithm designed specifically for time-sharing systems. It allocates a fixed time unit per process, known as a **Time Quantum** (TQ), ensuring that every process gets an equal share of the CPU.

Working Mechanism

1. All processes are treated as a circular queue.
2. The CPU scheduler assigns the CPU to each process for a duration equal to the **Time Quantum**.
3. If the process finishes within the TQ, it leaves the system.
4. If the process requires more time than the TQ, it is preempted at the end of the time slice and moved to the back of the queue.
5. This cycle continues until all processes are executed.

Selection of Time Quantum

The performance of Round Robin depends heavily on the size of the Time Quantum:

- If TQ is **very large**, Round Robin behaves like **FCFS**.
- If TQ is **very small**, it results in high **Context Switching Overhead**, which degrades system efficiency.

Calculations

The metrics are calculated as follows:

$$TAT = CT - AT \tag{5}$$

$$WT = TAT - BT \tag{6}$$

Program Code

```c
#include <stdio.h>

struct Process
{
    int pid, at, bt, rt, ct, tat, wt;
};

int main()
{
    int n, tq, current_time = 0, completed = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    struct Process p[n];
    int queue[100], front = 0, rear = 0;
    int visited[100] = {0};

    for (int i = 0; i < n; i++)
    {
        p[i].pid = i + 1;
        printf("Enter arrival and burst time for P%d: ", p[i].pid);
        scanf("%d %d", &p[i].at, &p[i].bt);
        p[i].rt = p[i].bt;
    }

    printf("Enter Time Quantum: ");
    scanf("%d", &tq);
```

Figure 9: Implemenation of Round Robin Algorithm Source Code 1

```
// Initial push: find process arriving at time 0
for (int i = 0; i < n; i++)
{
    if (p[i].at <= current_time)
    {
        queue[rear++] = i;
        visited[i] = 1;
    }
}

printf("\nGantt Chart:\n|");
while (completed != n)
{
    if (front == rear)
    { // No process in queue
        current_time++;
        for (int i = 0; i < n; i++)
        {
            if (p[i].at <= current_time && !visited[i])
            {
                queue[rear++] = i;
                visited[i] = 1;
            }
        }
        continue;
    }

    int i = queue[front++];
    printf(" P%d |", p[i].pid);

    if (p[i].rt > tq)
    {
        p[i].rt -= tq;
        current_time += tq;
    }
```

Figure 10: Implemenation of Round Robin Algorithm Source Code 2

```
        else
        {
            current_time += p[i].rt;
            p[i].rt = 0;
            completed++;
            p[i].ct = current_time;
            p[i].tat = p[i].ct - p[i].at;
            p[i].wt = p[i].tat - p[i].bt;
        }

        // Add newly arrived processes to queue
        for (int j = 0; j < n; j++)
        {
            if (p[j].at <= current_time && !visited[j])
            {
                queue[rear++] = j;
                visited[j] = 1;
            }
        }

        // If current process not finished, put it back in queue
        if (p[i].rt > 0)
        {
            queue[rear++] = i;
        }
    }

    printf("\n\nPID\tAT\tBT\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++)
    {
        printf("%d\t%d\t%d\t%d\t%d\t%d\n", p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);
    }
    return 0;
}
```

Figure 11: Implementation of Round Robin Algorithm Source Code 3

```
jatin@JatinWadhikarmi:/mnt/c/Users/jatin/OneDrive/Desktop/COMP 307 (OS)/Programs$ cd "/mnt/c/Users/jatin/OneDrive/Desktop/COMP 307 (OS)/Programs/" && gcc RoundRobin.c -o RoundRobin && "/mnt/c/Users/jat
in/OneDrive/Desktop/COMP 307 (OS)/Programs/"RoundRobin
Enter number of processes: 4
Enter arrival and burst time for P1: 0 3
Enter arrival and burst time for P2: 1 2
Enter arrival and burst time for P3: 2 1
Enter arrival and burst time for P4: 4 5
Enter Time Quantum: 2

Gantt Chart:
| P1 | P2 | P3 | P1 | P4 | P4 | P4 |

PID     AT      BT      CT      TAT     WT
1       0       3       6       6       3
2       1       2       4       3       1
3       2       1       5       3       2
4       4       5       11      7       2
```

Figure 12: Output of Round Robin Algorithm